COMP3011 Web Services and Web Data

API Documentation

COMP3011 Web Services and Web Data

Contents

1 Introduction **3**
- 1.1 Base URL . . . 3
- 1.2 API Versioning . . . 3
- 1.3 Content Type . . . 3

2 Authentication **3**
- 2.1 Token Format . . . 3
- 2.2 Token Lifecycle . . . 3
- 2.3 Obtaining Tokens . . . 3
- 2.4 Refreshing Tokens . . . 4

3 Rate Limiting **4**

4 Error Handling **4**
- 4.1 HTTP Status Codes . . . 4
- 4.2 Error Response Format . . . 4

5 Endpoints Reference **5**
- 5.1 Authentication Endpoints . . . 5
  - 5.1.1 Register User . . . 5
- 5.2 Task Endpoints . . . 6
  - 5.2.1 Task Object . . . 6
  - 5.2.2 Create Task . . . 6
  - 5.2.3 Query Parameters for List . . . 7
- 5.3 Category Endpoints . . . 7
- 5.4 Tag Endpoints . . . 7
- 5.5 Analytics Endpoints . . . 7
  - 5.5.1 Dashboard Response . . . 7
- 5.6 MCP (Model Context Protocol) Endpoints . . . 8
  - 5.6.1 Available Tools . . . 8
  - 5.6.2 Execute Tool . . . 8

6 Data Models **9**
- 6.1 Priority Levels . . . 9
- 6.2 Task Status . . . 9
- 6.3 Energy Levels . . . 9

7 Examples **10**
- 7.1 Complete Workflow Example . . . 10

7.1.1 1. Register and Authenticate 10
7.1.2 2. Create and Manage Tasks 10
7.1.3 3. Check Analytics 10

8 OpenAPI Specification 10

1 Introduction

This document provides comprehensive documentation for the Smart Productivity Analytics Platform API. The API enables task management and productivity analytics through a RESTful interface.

1.1 Base URL

```
https://your-domain.com/api/v1/
```

For local development:

```
http://localhost:8000/api/v1/
```

1.2 API Versioning

The API uses URL-based versioning. All endpoints are prefixed with `/api/v1/`.

1.3 Content Type

All requests and responses use JSON format:

```
Content-Type: application/json
```

2 Authentication

The API uses JWT (JSON Web Token) authentication. Tokens must be included in the `Authorization` header.

2.1 Token Format

```
Authorization: Bearer <access_token>
```

2.2 Token Lifecycle

Token Type	Lifetime	Purpose
Access Token	60 minutes	API request authentication
Refresh Token	7 days	Obtain new access tokens

Table 1: JWT Token Configuration

2.3 Obtaining Tokens

Endpoint: `POST /api/v1/auth/token/`

Request:

```
{
    "email": "user@example.com",
    "password": "your_password"
}
```

Response (200 OK):

```
{
    "access": "eyJ0eXAiOiJKV1QiLCJhbGc...",
    "refresh": "eyJ0eXAiOiJKV1QiLCJhbGc..."
}
```

2.4 Refreshing Tokens

Endpoint: `POST /api/v1/auth/token/refresh/`

Request:

```
{
    "refresh": "<refresh_token>"
}
```

3 Rate Limiting

User Type	Limit
Anonymous	100 requests/hour
Authenticated	1000 requests/hour

Table 2: Rate Limits

When rate limited, the API returns HTTP 429 with a `Retry-After` header.

4 Error Handling

4.1 HTTP Status Codes

Code	Status	Description
200	OK	Request successful
201	Created	Resource created successfully
204	No Content	Resource deleted successfully
400	Bad Request	Invalid request data
401	Unauthorized	Authentication required
403	Forbidden	Permission denied
404	Not Found	Resource not found
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server error

Table 3: HTTP Status Codes

4.2 Error Response Format

```
{
    "detail": "Error message describing the issue",
    "code": "error_code"
}
```

5 Endpoints Reference

5.1 Authentication Endpoints

Method	Endpoint	Description
POST	/auth/register/	Register a new user account
POST	/auth/token/	Obtain JWT token pair
POST	/auth/token/refresh/	Refresh access token
POST	/auth/token/blacklist/	Invalidate refresh token (logout)
GET	/auth/profile/	Get current user profile
PUT/PATCH	/auth/profile/	Update user profile
POST	/auth/change-password/	Change password

Table 4: Authentication Endpoints

5.1.1 Register User

POST `/api/v1/auth/register/`

Request Body:

```
{
    "email": "user@example.com",
    "username": "johndoe",
    "password": "SecurePassword123!",
    "password_confirm": "SecurePassword123!"
}
```

Response (201 Created):

```
{
    "user": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "email": "user@example.com",
        "username": "johndoe"
    },
    "tokens": {
        "access": "eyJ0eXAiOiJKV1Q...",
        "refresh": "eyJ0eXAiOiJKV1Q..."
    },
    "message": "Registration successful."
}
```

5.2 Task Endpoints

Method	Endpoint	Description
GET	/tasks/	List all tasks (with filtering)
POST	/tasks/	Create a new task
GET	/tasks/{id}/	Get task details
PUT	/tasks/{id}/	Update task (full)
PATCH	/tasks/{id}/	Update task (partial)
DELETE	/tasks/{id}/	Delete task
POST	/tasks/{id}/complete/	Mark task as completed
POST	/tasks/{id}/reopen/	Reopen completed task
GET	/tasks/today/	Get today's tasks
GET	/tasks/overdue/	Get overdue tasks
POST	/tasks/bulk_update_status/	Bulk update task status

Table 5: Task Endpoints

5.2.1 Task Object

```
{
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "title": "Complete project report",
    "description": "Write the final section",
    "status": "pending",
    "priority": 4,
    "energy_level": "high",
    "due_date": "2026-03-15T17:00:00Z",
    "category": {
        "id": "...",
        "name": "Work"
    },
    "tags": [
        {"id": "...", "name": "urgent"}
    ],
    "created_at": "2026-03-10T10:00:00Z",
    "updated_at": "2026-03-10T10:00:00Z"
}
```

5.2.2 Create Task

POST `/api/v1/tasks/`

Request Body:

```
{
    "title": "Complete project report",
    "description": "Write the final section",
    "priority": 4,
    "energy_level": "high",
    "due_date": "2026-03-15T17:00:00Z",
    "category": "550e8400-e29b-41d4-a716-446655440000",
    "tags": ["550e8400-e29b-41d4-a716-446655440001"]
}
```

5.2.3 Query Parameters for List

Parameter	Type	Description
status	string	Filter by status (pending, in_progress, completed, cancelled)
priority	integer	Filter by priority (1-5)
category	UUID	Filter by category ID
energy_level	string	Filter by energy level (low, medium, high)
search	string	Search in title and description
ordering	string	Sort field (e.g., -due_date, priority)

Table 6: Task Query Parameters

5.3 Category Endpoints

Method	Endpoint	Description
GET	/categories/	List all categories
POST	/categories/	Create a category
GET	/categories/{id}/	Get category details
PUT/PATCH	/categories/{id}/	Update category
DELETE	/categories/{id}/	Delete category
GET	/categories/{id}/stats/	Get category statistics

Table 7: Category Endpoints

5.4 Tag Endpoints

Method	Endpoint	Description
GET	/tags/	List all tags
POST	/tags/	Create a tag
GET	/tags/{id}/	Get tag details
PUT/PATCH	/tags/{id}/	Update tag
DELETE	/tags/{id}/	Delete tag

Table 8: Tag Endpoints

5.5 Analytics Endpoints

Method	Endpoint	Description
GET	/analytics/dashboard/	Dashboard overview
GET	/analytics/tasks/	Task analytics
GET	/analytics/trends/	Productivity trends

Table 9: Analytics Endpoints

5.5.1 Dashboard Response

```
{
```

```
    "today": {
        "tasks_created": 5,
        "tasks_completed": 3
    },
    "week": {
        "tasks_created": 25,
        "tasks_completed": 18,
        "completion_rate": 72.0
    },
    "recent_tasks": [
        {
            "id": "...",
            "title": "Write report",
            "status": "completed"
        }
    ]
}
```

5.6 MCP (Model Context Protocol) Endpoints

MCP endpoints enable AI agent integration with standardized tool discovery and execution.

Method	Endpoint	Description
GET	/mcp/capabilities/	Get API capabilities (public)
GET	/mcp/tools/	List available tools
POST	/mcp/execute/	Execute a tool

Table 10: MCP Endpoints

5.6.1 Available Tools

Tool Name	Description
create_task	Create a new task
list_tasks	List tasks with filtering
complete_task	Mark a task as completed
get_today_tasks	Get tasks due today
get_overdue_tasks	Get overdue tasks
suggest_next_task	Get AI-powered task suggestion based on energy level
get_productivity_summary	Get productivity metrics
create_category	Create a new category

Table 11: MCP Tools

5.6.2 Execute Tool

POST `/api/v1/mcp/execute/`

Request Body:

```
{
    "tool": "suggest_next_task",
    "arguments": {
        "energy_level": "low"
    }
```

```
}
```

Response (200 OK):

```
{
    "success": true,
    "tool": "suggest_next_task",
    "result": {
        "task": {
            "id": "...",
            "title": "Organize documents",
            "priority": 2
        },
        "reason": "Low-energy task suitable for current state"
    }
}
```

6 Data Models

6.1 Priority Levels

Value	Label
1	Lowest
2	Low
3	Medium
4	High
5	Urgent

Table 12: Task Priority Levels

6.2 Task Status

Value	Description
pending	Task not yet started
in_progress	Task is being worked on
completed	Task is finished
cancelled	Task was cancelled

Table 13: Task Status Values

6.3 Energy Levels

Value	Suitable For
low	Administrative tasks, organizing
medium	Standard work tasks
high	Complex problem-solving, creative work

Table 14: Energy Level Values

7 Examples

7.1 Complete Workflow Example

7.1.1 1. Register and Authenticate

```
# Register
curl -X POST http://localhost:8000/api/v1/auth/register/ \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","username":"john",
       "password":"SecurePass123!","password_confirm":"SecurePass123!"}'

# Login
curl -X POST http://localhost:8000/api/v1/auth/token/ \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"SecurePass123!"}'
```

7.1.2 2. Create and Manage Tasks

```
# Create task
curl -X POST http://localhost:8000/api/v1/tasks/ \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"title":"Write report","priority":4,"due_date":"2026-03-15T17
     :00:00Z"}'

# Complete task
curl -X POST http://localhost:8000/api/v1/tasks/<id>/complete/ \
  -H "Authorization: Bearer <token>"
```

7.1.3 3. Check Analytics

```
# Get dashboard
curl -X GET http://localhost:8000/api/v1/analytics/dashboard/ \
  -H "Authorization: Bearer <token>"

# Get productivity trends
curl -X GET http://localhost:8000/api/v1/analytics/trends/?period=week \
  -H "Authorization: Bearer <token>"
```

8 OpenAPI Specification

The complete OpenAPI 3.0 specification is available at:

- **Interactive (Swagger UI):** `/api/v1/docs/swagger/`
- **ReDoc:** `/api/v1/docs/redoc/`
- **JSON Schema:** `/api/v1/docs/schema/`
- **Repository:** https://github.com/Ejkenway/COMP3011-CW1/blob/main/todo_api/openapi.yaml